Circuit Simulator in C++

Code Review of the OOP Course Project (Phases 0–2)

Student	Seyed Kiarash Eshraghi Naeini (403101383), team of two
Instructor	Dr. Abdollah Arasteh
Semester	Spring 2025 (second semester 1403–04) review written: October 2026
Stack	C++17/20, Eigen 3.4 (MNA solver), Qt 6 Widgets + Charts + Network, cereal (serialization)

Summary

The course project was a SPICE-like circuit simulator built in three phases: an object-oriented design (Phase 0), a command-line simulator based on modified nodal analysis (Phase 1) and a graphical version with schematic entry, waveform viewer, project files and networking (Phase 2). It was a two-person group project. This document reviews the *final* code of each phase as it was submitted; nothing was fixed or completed for this review. Both programs were rebuilt from source and exercised: the Phase 1 simulator with scripted test circuits, cross-checked against `ngspice`, and the Phase 2 application under a virtual display with its analysis engine also tested on its own.

In short, the Phase 1 linear solver is correct where it is complete. Resistors, sources, the four dependent sources it supports and backward-Euler transients of RC, RL and RLC circuits agree with `ngspice` to the expected first-order accuracy. Several features are only partly working: current-controlled voltage sources, inductors in DC analysis, ideal diodes and the ground command (Section 3.5). The Phase 2 application has a clean MVC layout and a capable plotter, but in the final version the schematic, the solver and the plot window are not connected end to end (Section 4.5). Final code: `https://github.com/seyedkiarash/seyedkiarash.github.io/tree/main/projects/oop-circuit-simulator`.

1 Project timeline and scope

Table 1. Phases, deliverables and the code reviewed.

Phase	Deliverable	Final code reviewed
0	Object-oriented design: classes, UML, design patterns, numerical methods, exception design (individual report)	design document, Apr 2025
1	Console simulator: LTspice-style commands, MNA with Eigen, transient and DC-sweep analysis, file menu	1 430 lines, one file, 10 Jun 2025
2	Graphical simulator: schematic editor, waveform viewer with cursors and math, AC/phase sweeps, sub-circuits, serialization, TCP networking	5,401 lines in 48 files, 4 Sep 2025

The Phase 2 handout suggested SDL for the user interface; the team used Qt 6 instead (widgets, `QGraphicsView` for the schematic, Qt Charts for plots and `QTcpSocket` for networking). Where several copies of the code existed (backup folders, an archive and three GitHub repositories), the file modification dates were compared and the newest complete copy was taken as final. For Phase 1 that is the working copy of the team repository, which is newer than its last commit. For Phase 2 it is the `backup` folder, last edited on 4 September 2025.

2 Phase 0 – design intent

The design report proposed a `Node` class, an abstract `Element` base with one derived class per component, and a `Circuit` class owning both. It also proposed five patterns and techniques:

- a *Factory Method* for creating elements from parsed commands;
- a *Bridge* between the circuit and interchangeable linear solvers (Gaussian elimination, LU or a library such as Eigen);
- *MVC* for separating analysis from presentation;

- the *Command* pattern for the console interface;
- an exception hierarchy (`SpiceException` with parsing, circuit and analysis subclasses).

Table 4 at the end compares this plan with what was built.

3 Phase 1 – console simulator

3.1 Architecture

The program is one translation unit organised in three marked sections, *Models*, *Controller* and *View* (Fig. 1). Elements derive from an abstract `Element` that holds two `Node*` and exposes virtual getters. `MakingMNA` assembles the MNA system and `MNASolver` solves it with Eigen's partial-pivot LU. `TransientAnalysis` and `DCSweepAnalysis` drive the solver over time or over a source value. The `centralController` owns nodes and elements and implements every command, and `CommandParser` maps input lines to controller calls with one regular expression per command.

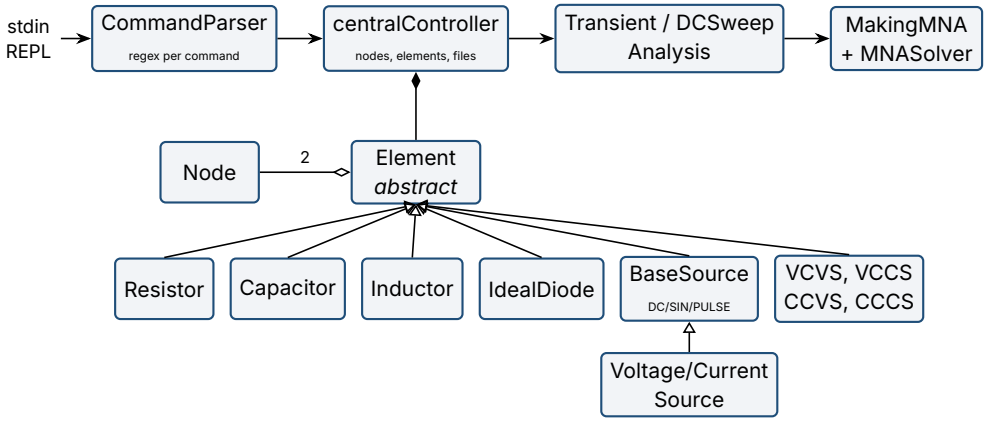


Figure 1. Phase 1 class structure (inheritance: open arrows; ownership: diamonds).

3.2 Numerical method

With n non-ground nodes, the unknown vector holds the node voltages followed by one branch current for each voltage source, inductor, diode, VCVS and CCVS. Each element adds ("stamps") a few entries into $A\,x = z$. Capacitors and inductors use the backward-Euler companion models

$$i_C^{k+1} = \frac{C}{h}\left(v^{k+1} - v^{k}\right), \qquad v_L^{k+1} - \frac{L}{h}\,i_L^{k+1} = -\frac{L}{h}\,i_L^{k},$$

so a capacitor is a conductance C/h in parallel with a current source $C v^k/h$, and an inductor is an extra row with $-L/h$ on the diagonal. The matrix is rebuilt and factorised at every time step. The signs follow SPICE conventions: a source current is positive when it flows into the positive terminal, and VCCS and CCCS currents flow from n_+ to n_- through the source.

3.3 Command interface

Supported commands are listed below. Values accept SPICE suffixes (`k`, `meg`, `m`, `u`, `n`, `p`) and scientific notation.

- `add R|C|L|D|V|I|E|G|H|F …`: add an element. Sources take a DC value, `SIN(off amp f)` or SPICE-style `PULSE(v1 v2 td tr tf pw per)`.
- `delete` *name*: remove an element. Deletion is refused if the element is a control branch of a dependent source, and unused nodes are cleaned up.
- `nodes`, `list [type]` and `rename node` *old new*.
- `.TRAN` and `.DC` define an analysis; `print TRAN|DC … V(`*n*`) I(`*e*`)` runs it.
- `show existing schematics`, `NewFile` *name*, a number to load a schematic, and `return`.

3.4 Verification

The final source compiles without warnings (`g++ -std=c++17 -Wall -Wextra`). Results were compared with closed-form solutions and with `ngspice`-42 (Table 2).

Table 2. Phase 1 test circuits.

Test	Result	Observation
Divider, DC sweep $0 \to 5$ V; V, $I(R)$, $I(V)$	**PASS**	exact; source current has SPICE sign
VCVS, VCCS, CCCS with resistive loads	**PASS**	identical to ngspice $(3, -1, 2$ V; -1 mA$)$
RC step, $h = \tau/10 \ldots \tau/500$	**PASS**	first-order convergence (Fig. 3)
Series RLC step, $\zeta = 0.16$	**PASS**	matches ngspice (Fig. 2)
RL step, inductor current	**PASS**	backward-Euler values
Input validation (negative/zero values, duplicate names, lower-case prefixes)	**PASS**	rejected with an error message
CCVS with grounded output terminal	**FAIL**	exception `map::at`
CCVS with floating output	**FAIL**	"singular matrix"
Inductor in DC sweep	**FAIL**	"singular matrix"
Diode forward-biased by 5 V	**FAIL**	output stays 0 V
`add GND b` on a circuit without a node named 0/GND	**FAIL**	"ground not detected"
Four-resistor ladder, $R = 100\,\Omega$	**FAIL**	false "singular matrix" (1 and 10 kΩ pass)
Values `10uF`, `5mH`; `SIN(1, 2.5, 50)`	**FAIL**	rejected (unit letter, commas)
Save and reload: `4.7n` capacitor; SIN source	**FAIL**	saved as `0.000000`; SIN saved as DC

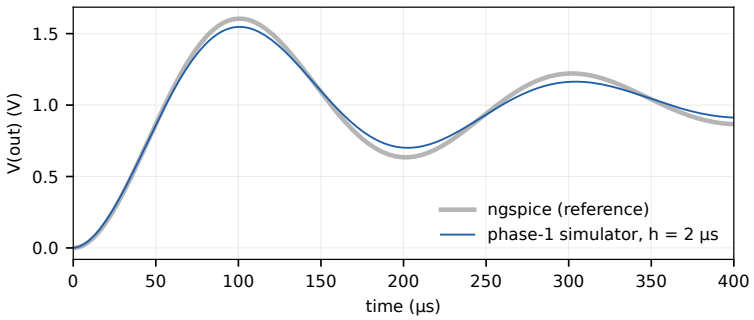


Figure 2. Series RLC step response ($R = 10\,\Omega$, $L = 1$ mH, $C = 1\,\mu$F).

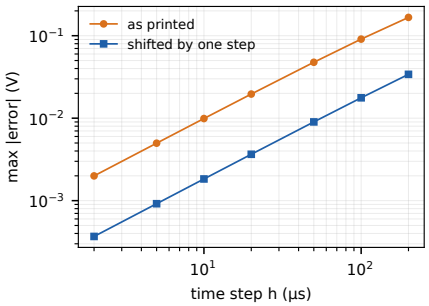


Figure 3. RC step error versus time step.

The error falls linearly with h, as expected for backward Euler. The printed error, however, is about five times larger than the method's own error. The reason is that the row labelled t_k actually contains the solution at t_{k+1}: the loop solves first and then prints the old time. Shifting the time column by one step leaves only the integration error (blue curve in Fig. 3).

3.5 Findings

Severity labels: **[H]** wrong results or a crash, **[M]** a missing or inaccurate feature, **[L]** style or usability.

1. **[H] CCVS stamp is incomplete.** The CCVS row is written, but its branch current is never added to the KCL equations of its two nodes, so that column of A is zero and the matrix is singular. For a grounded terminal the code also calls `nodeToIndexMap.at()` on the ground node, which is not in the map and throws.
2. **[H] Inductors in DC analysis.** In DC mode only the row $v_1 - v_2 = 0$ is stamped. The current column is left out of KCL, so any circuit with an inductor fails the DC sweep.
3. **[H] Ideal diodes never conduct.** Diodes start `OFF` and nothing changes their state. An earlier commit (6 June) iterated the ON/OFF states at each step and also had an adaptive time step. Neither survived the merge into the final file.
4. **[M] `add GND` *node* has no effect.** It only creates the node. Ground is recognised by name alone (`0`, `GND`, `gnd`).
5. **[M] Singularity test depends on scale.** A matrix is rejected when $|\det A| < 10^{-12}$. The determinant shrinks as G^n, so ordinary high-impedance circuits are rejected. A rank or condition-number check on the LU factors would be scale-independent.
6. **[M] Saving loses information.** Values are written with `std::to_string` (six decimals), so a 4.7 nF capacitor becomes `0.000000` and is dropped on reload, and 2.2 μF becomes 2 μF. SIN and PULSE sources are saved as their instantaneous value.
7. **[M] Units and syntax.** Suffixes followed by a unit (`uF`, `nF`, `mH`) are rejected. A bare `f` is read as farad, whereas SPICE reads it as femto. The dotted spellings in the handout (`.print`, `.nodes`, `.list`, `.rename`) are

not accepted, and SIN arguments separated by commas fail, although the team's own test plan expected them to work.

8. **[M] Transient options.** Output is shifted by one step (see above). Initial conditions are all zero, with no DC operating point. The optional `Tstart` and `Tmaxstep` are parsed but ignored. `I(C)` always prints 0 because `Capacitor` does not override `getCurrent()`.
9. **[L] DC sweep side effect.** Sweeping a source calls `setValue()`, which permanently turns a SIN or PULSE source into a DC source.
10. **[L] Design.** Every analysis step uses long `dynamic_cast` chains instead of a virtual `stamp()` on each element, so adding an element type means editing the matrix builder, the vector builder and the state update (not open for extension). Ownership is manual `new/delete`. Twenty-odd `std::regex` objects are compiled for every input line. The dense $O(n^3)$ factorisation is repeated each step although A only changes when h or a diode state changes.

Strengths: the separation into model, controller and view is clear even within one file. Validation messages follow the handout. Errors are thrown and caught at the REPL, so a bad command never ends the session. Deletion protects control branches of dependent sources. Sign conventions agree with SPICE throughout.

4 Phase 2 – graphical simulator

4.1 Architecture

The code is split into `Core`, `Model`, `Controller`, `View` and `Network` folders (Fig. 4).

- **Model.** `Project` holds `shared_ptrs` to elements and nodes plus the simulation settings, and is serialised polymorphically with cereal. `CircuitModel` translates a project into a new, struct-based `AnalysisEngine` (transient, AC and phase sweeps).
- **Controller.** `ApplicationController` saves and loads projects and sub-circuits and reports to the view through Qt signals.
- **View.** `GraphicsView` is the schematic editor, split into placement, wiring and editing files. Each component type has a `CircuitComponent` subclass that draws its symbol. `Plotter` reads result files in a worker thread and adds curves chunk by chunk.

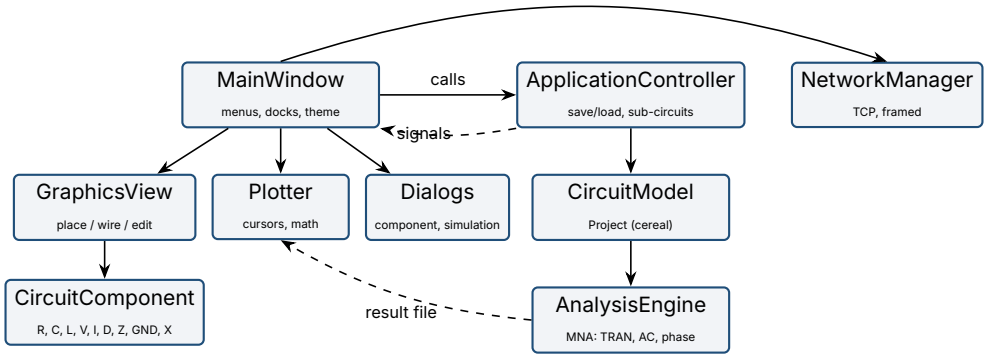


Figure 4. Phase 2 structure. The dashed result-file path existed in the 20 August version and is not used by the final menu (finding 2).

4.2 Building and running

The final sources build with CMake, Qt 6.4 and system Eigen and cereal. The one change needed was the `QDataStream::Qt_6_7` constant in the network code, which requires Qt 6.7 or newer. The application crashed when the first component was placed (finding 1). The screenshots below therefore come from a test build in which three pointer members were initialised to `nullptr`. That change is not in the published code.

4.3 Analysis engine

Because the menu no longer calls the engine, it was tested directly with a small harness. The transient solver uses the same backward-Euler models as Phase 1. For an RC step with $h = \tau/100$, it agrees with $1-e^{-t/\tau}$ to within 1% of the step (largest error at $t = 0$, from the same one-step offset). The AC solver builds a complex

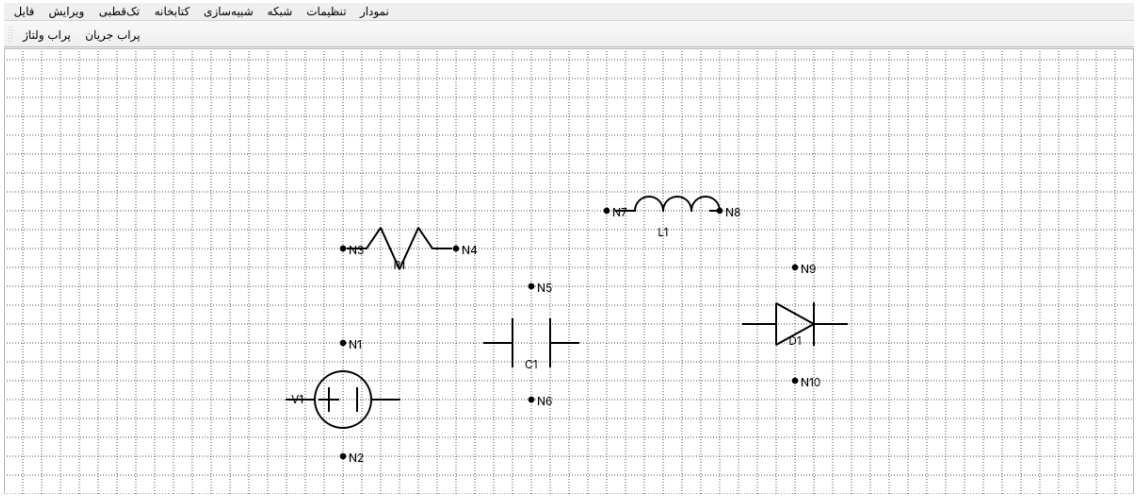


Figure 5. Schematic editor: a source, resistor, capacitor, inductor and diode placed with keyboard shortcuts; the source, capacitor and diode were rotated by 90°. Each terminal gets its own node name (N1, N2, …).

MNA system with admittances $j\omega C$ and $1/(j\omega L)$, and its phasors are correct. However, the engine always reports the *first* non-ground node, which in a typical circuit is the source node (Fig. 7). The phase sweep is a stub: it runs the same AC point repeatedly without applying the phase, so its output is constant.

4.4 Feature coverage

Table 3 follows the course's scoring sheet. "Partial" means the code exists but the feature does not work end to end in the final version.

Table 3. Phase 2 requirements and their state in the final code.

Requirement	State	Note
Graphical elements, shortcuts, grid snapping, rotation	**PASS**	R, C, L, V, I, D, Z, GND, sub-circuit symbols
Node labels and renaming; same-name nodes are joined	**PASS**	names are what connect a circuit
Wires connect elements without explicit nodes	**PARTIAL**	wires are drawn but carry no connection
Transient analysis from the schematic	**PARTIAL**	engine works; menu shows sample traces
AC sweep	**PARTIAL**	first node only; linear sweep only
Phase sweep	**FAIL**	stub
Probes (voltage/current/power)	**PARTIAL**	cursor graphics only
Several signals, colours, rename, math $(+, -, \times, \div)$	**PASS**	math uses samples at matching times
Horizontal/vertical scaling, zoom, auto-zoom	**PASS**	menus and toolbar
Cursor and double cursor $(\Delta x, \Delta y$, slope)	**PASS**	
Sub-circuits: create, choose pins, save to library, reuse	**PARTIAL**	stored and drawn, not simulated
Thevenin/Norton equivalent of a sub-circuit	**FAIL**	not implemented
Waveform (sampled) source	**FAIL**	a class exists but is not built or used
Project save/load with object serialization	**PASS**	cereal binary; round trip verified
Networking: send data, remote source, link two circuits	**PARTIAL**	sends one element; no remote source
Name, icon, dark theme	**PASS**	

4.5 Findings

1. **[H] Uninitialised pointers.** `previewGraphics`, `previewWire` and `circuitModel` in `GraphicsView` have no initialiser. `startPlacing()` first calls `cancelPlacing()`, which deletes `previewGraphics` if it is non-null. With a garbage value this is undefined behaviour. On Linux it crashes (segmentation fault in `QGraphicsScene::removeItem`); on the team's Windows build heap memory was probably zeroed, which hid the bug.
2. **[H] The run command is disconnected.** `on_actionRunAnalysis_triggered()` now only calls `showDefault-`

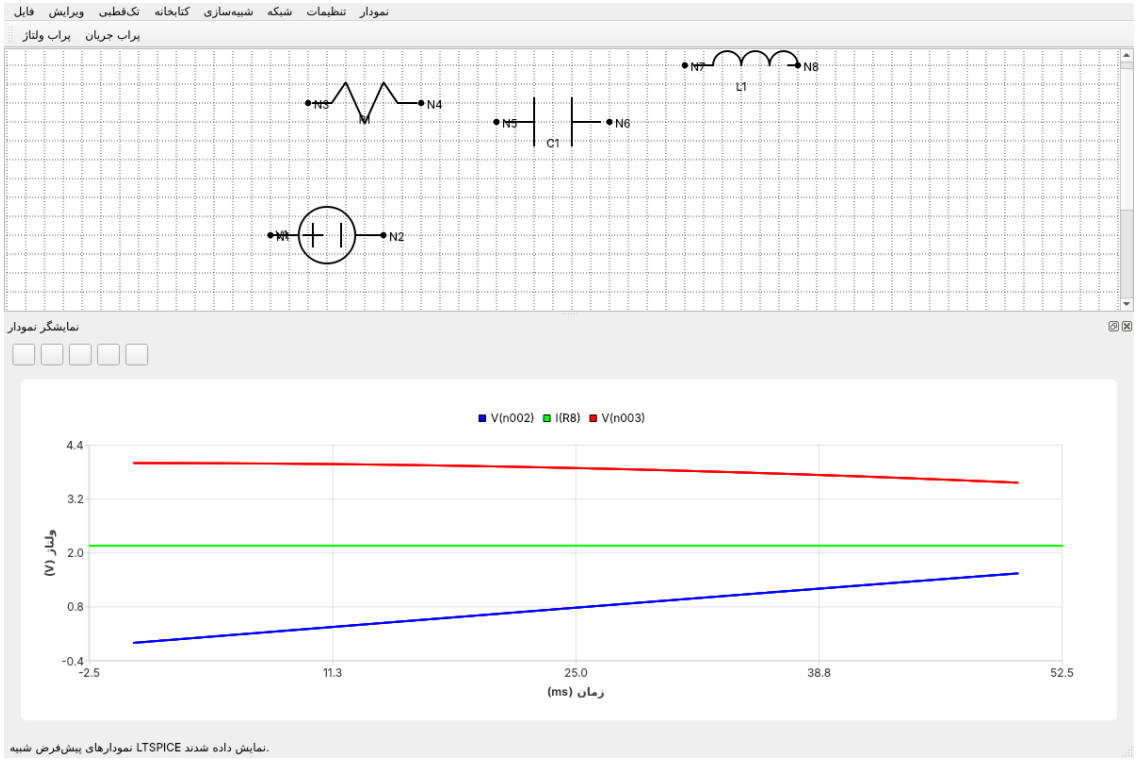


Figure 6. After *Simulation → Settings and run* the waveform viewer shows three built-in sample traces (labelled `V(n002)`, `I(R8)`, `V(n003)`), not results of the drawn circuit.

`Charts()`, which draws synthetic sine and DC traces. The 20 August version opened `SimulationDialog` and passed the settings to `ApplicationController::runAnalysis`, whose output file `Plotter` then loaded.

3. **[H] Connectivity by name only.** Each placed element and each wire segment gets new, uniquely named nodes, and the engine ignores `Wire` elements. Terminals therefore connect only when their nodes have the same name: ground symbols (all named `0`) or nodes renamed by the user. Merging coincident terminals and wire endpoints (e.g. with union–find) before building the netlist is the missing step.
4. **[M] Engine coverage.** Diodes, Zener diodes, sub-circuits and PULSE sources are skipped (PULSE evaluates to its DC value, 0). AC analysis drives every source, DC sources included, with its value. The octave/decade sweep option is ignored. Two fallback branches return made-up data (a constant magnitude of 10, or the first source's voltage) instead of reporting an error. The dependent sources from Phase 1 were not carried over because the engine was rewritten instead of reused.
5. **[M] Serialization.** An earlier project file (`t1.proj`, 16 August) no longer loads ("failed to read 4 bytes"), most likely because the class layout changed and no cereal class versions are declared. Saving a subcircuit copies the project's `shared_ptrs`, so the definition aliases the live circuit until it is written to disk.
6. **[M] Networking.** Messages are length-prefixed (good), but the expected size is stored in a member shared by all client sockets. `waitForConnected` blocks the GUI thread for up to 3 s, and *Send data* transmits only the first element of the project.
7. **[L] View details.**
   - Rotation moves the terminals but draws the body unrotated (Fig. 5).
   - `ZenerDiodeView` is never created, because the `dynamic_cast<Diode*>` test is reached first.
   - The shortcuts `Ctrl+C`, `Ctrl+V` and `Ctrl+Z` override copy, paste and undo.
   - Toolbar icons come from `QIcon::fromTheme`, which is empty on Windows, so the buttons are blank.
   - In dark mode the symbols are drawn black on a dark background.
8. **[L] Dead code.** Eleven root-level files are not in `CMakeLists.txt`: `waveformsource`, `componentlibrary`, `nodelabel`, `subcircuitcomponent` and `wire` (header and source each), and `main2.cpp`, a copy of the Phase 1 program.

Strengths: the folder-level MVC split is consistent, and the view talks to the controller only through methods and Qt signals. Element ownership uses `shared_ptr`, and cereal's polymorphic registration gives one-line project save/load; a saved project (`t5.proj`) loads and round-trips correctly. The plotter is the most complete part. It reads data in a background thread in chunks, as the handout recommends for long simulations, and

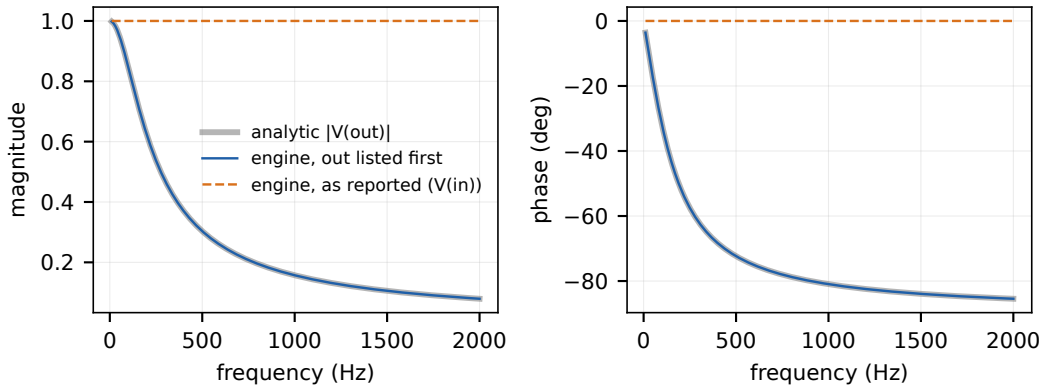


Figure 7. AC sweep of an RC low-pass ($f_c = 159\,\mathrm{Hz}$). The solution is correct when the output node happens to be listed first (blue); as written, the engine reports the source node, whose magnitude is 1 (orange).

offers auto-zoom, independent axis scaling, colour and name changes, math traces and single and double cursors.

5 Design plan versus implementation

Table 4. Phase 0 proposals and how they appear in the code.

Proposal	Implementation
Abstract `Element`, polymorphism	Used in both phases; behaviour is selected with `dynamic_cast` rather than virtual methods
Factory Method for parsing	`CommandParser` plus one `making…()` method per element (Phase 1); `startPlacing()` maps type names to classes (Phase 2)
Bridge for linear solvers	Not used; Eigen `PartialPivLU` is called directly
MVC	Sections in Phase 1; folders, a controller and Qt signals in Phase 2
Command pattern for the console	A regex dispatcher with direct calls, not command objects
`SpiceException` hierarchy	Standard `logic_error`/`runtime_error` with descriptive messages
Euler or Runge–Kutta integration	Backward Euler (implicit, stable for stiff RC circuits); adaptive step tried, then dropped

6 Lessons

- Give each element a virtual `stamp(A, z, h)` method. The solver then never needs to know element types, and Phase 2 could have reused the Phase 1 engine, dependent sources included.
- Build the netlist from geometry: merge terminals that touch and the endpoints of wires, then name the nets.
- Keep a small set of golden circuits with reference results (here from ngspice) and run them on every change. Most findings above are regressions that such tests catch immediately.
- Initialise every pointer member and build on a second platform; uninitialised members often work on one compiler and crash on another.

How this review was done

Phase 1 was compiled with `g++` 13.3 and Eigen 3.4 and driven through standard input by scripts. Reference results came from `ngspice`-42 and closed-form solutions. Phase 2 was built with CMake, Qt 6.4.2 and cereal, run under Xvfb with input from `xdotool`, and debugged with `gdb`. Its analysis engine and serialization were tested with small C++ harnesses linked against the original source files. The published code is the final submitted version, without third-party libraries and build folders: `https://github.com/seyedkiarash/seyedkiarash.github.io/tree/main/projects/oop-circuit-simulator`.